

# Design & Analysis of Algorithms

## Lab 1: Algorithm Performance Analysis Workbench

Name: Manas Bhasker

Roll No: 2501010226

Email: 2501010226@krmu.edu.in

---

### Q1. 1.2 Linear Search and Binary Search Comparison (5 marks)

```
def linear_search(arr, target):
    c = 0
    for i in range(len(arr)):
        c += 1
        if arr[i] == target:
            return i, c
    return -1, c

def binary_search(arr, target):
    l = c = 0
    h = len(arr) - 1
    while l <= h:
        mid = (l+h)//2
        c += 1
        elem = arr[mid]
        if elem == target:
            if elem in arr[:mid]:
                h = mid - 1
                continue
            else:
                return mid, c
        if elem < target:
            l = mid + 1
        else:
            h = mid - 1
    return -1, c

def compare_search_algorithms(arr, target):
    output = []
    i1, c1 = linear_search(arr, target)
    i2, c2 = binary_search(arr, target)
    output.append("Search Comparison Report")
    output.append("Linear Search")
    output.append(f"Index: {i1}")
    output.append(f"Comparisons: {c1}")
    output.append("Binary Search")
    output.append(f"Index: {i2}")
    output.append(f"Comparisons: {c2}")
    if c1 < c2:
        output.append("Better Algorithm: Linear Search")
    else:
        output.append("Better Algorithm: Binary Search")
    return output
```

### Q2. 1.1 Recursive and Iterative Computation Analyzer (10 marks)

```
def rec_fac(n, c=0):
    c += 1
    if n == 0:
        return 1, c
    fac, c = rec_fac(n-1, c)
    return n * fac, c

def iter_fac(n):
    c = 0
    fac = 1
    for i in range(1, n+1):
        c += 1
        fac *= i
    return fac, c

def rec_fib(n, c=0):
    c += 1
    if n == 0 or n == 1:
        return n, c
    l, c = rec_fib(n-2, c)
    r, c = rec_fib(n-1, c)
```

```

    return l + r, c

def iter_fib(n):
    c = 0
    a, b = 0, 1
    for i in range(1, n+1):
        c += 1
        a, b = b, a+b
    return a, c

def analyze_recursive_iterative(n):
    output = []
    fac1, fac_c1 = rec_fac(n)
    fac2, fac_c2 = iter_fac(n)
    fib1, fib_c1 = rec_fib(n)
    fib2, fib_c2 = iter_fib(n)
    output.append("Computation Analysis Report")
    output.append(f"Recursive Factorial: {fac1}")
    output.append(f"Iterative Factorial: {fac2}")
    output.append(f"Recursive Fibonacci: {fib1}")
    output.append(f"Iterative Fibonacci: {fib2}")
    output.append("Operation Count Comparison")
    output.append(f"Recursive Factorial Count: {fac_c1}")
    output.append(f"Iterative Factorial Count: {fac_c2}")
    output.append(f"Recursive Fibonacci Count: {fib_c1}")
    output.append(f"Iterative Fibonacci Count: {fib_c2}")
    return output

```

### Q3. 1.3 Bubble Sort and Insertion Sort Performance Comparison (10 marks)

```

def bubble_sort(arr):
    n = len(arr)
    s = c = 0
    for i in range(n-1):
        swapped = False
        for j in range(n-1-i):
            c += 1
            if arr[j] > arr[j+1]:
                s += 1
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return " ".join(map(str, arr)), s, c

def insertion_sort(arr):
    n = len(arr)
    i = 1
    s = c = 0
    while i < n:
        x = arr[i]
        j = i - 1
        while j >= 0:
            c += 1
            if arr[j] > x:
                s += 1
                arr[j+1] = arr[j]
                j -= 1
            else:
                break
        arr[j+1] = x
        i += 1
    return " ".join(map(str, arr)), s, c

def compare(dataset):
    output = []
    bs_sort, bs_s, bs_c = bubble_sort(dataset[:])
    is_sort, is_s, is_c = insertion_sort(dataset[:])
    output.append(f"Bubble Sorted: {bs_sort}")
    output.append(f"Bubble Comparisons: {bs_c}")
    output.append(f"Bubble Swaps: {bs_s}")
    output.append(f"Insertion Sorted: {is_sort}")
    output.append(f"Insertion Comparisons: {is_c}")
    output.append(f"Insertion Shifts: {is_s}")
    if bs_c < is_c:
        output.append("Better Algorithm: Bubble Sort")
    elif bs_c > is_c:
        output.append("Better Algorithm: Insertion Sort")

```

```
    else:
        output.append("Better Algorithm: Both Equal")
    return output

def compare_bubble_insertion(random_data, sorted_data, reverse_data):
    output = []
    output.append("Sorting Performance Report")
    output.append("Random Dataset")
    output.extend(compare(random_data))
    output.append("Sorted Dataset")
    output.extend(compare(sorted_data))
    output.append("Reverse Dataset")
    output.extend(compare(reverse_data))
    return output
```